LAMP STACK - LEARNING TASK

LAMP stack combines any Linux-based operating system, Apache as the web server, MySQL/MariaDB as the database system, and PHP/Python/Perl as the programming language.

sudo ufw allow ssh allows incoming traffic on the default SSH port (Port 22)
sudo ufw allow 80 allows incoming traffic on Port 80(HTTP)
sudo ufw allow 443 allows incoming traffic on Port 443(HTTPS)
sudo ufw enable enables the Uncomplicated Firewall (ufw) and activates the firewall

sudo apt install apache2 - installation

sudo apt install apache2
sudo yum install -y httpd

sudo systemctl status apache2 - to know whether it works

sudo apt install mysql-server
sudo yum install -y mysql-server
sudo systemctl start mysqld
sudo systemctl enable mysqld

sudo MySQL - to access.

You should replace this file (located at /var/www/html/index.html) before continuing to operate your HTTP server.

What is LAMP Stack?

LAMP Stack is a set of open source technologies used for web development consist of Linux as an operating system, Apache HTTP as a web server, MySQL as a database service & PHP as a programming language for developing the web applications.

Workflow:

1. It Receives requests from Apache webserver.
2. It processes request from PHP file may be the dynamic request that might requires to retrieve information from MySQL database.
3. Then it returns the calculated responses to the HTTP server & update the database. Then finally it displayed the request to the browser.

Uses:

Data processing
Database components
Business logic in code
API for communicating with other applications

Benefits:

1. Cost effective As it is a set of open source technologies
2. Efficiency for developing large scale applications
2. Support It has large global IT community.

sudo vi /etc/apache2/mods-available/mpm_prefork.conf -- can adjust max number of workers

To optimize keep alive settings:

. Edit Apache Configuration File:
sudo vi /etc/apache2/apache2.conf

. Adjust KeepAliveTimeout:
Find and adjust the KeepAliveTimeout
KeepAliveTimeout 5

KeepAlive:
When It's On, it means the server allows a single connection to be used for multiple requests from the same user.
Each request opens a new connection when it's off, like starting a new conversation every time.
Having KeepAlive On can improve performance by reducing the time it takes to set up a new connection for each request.

Apache configuration file which is located at /etc/apache2/apache2.conf, typically contains global settings that apply to the entire server.

MULTI PROCESSING MODULES:

mpm_event and mpm_worker are Multi-Processing Modules (MPMs) that handle the creation of child processes or threads to serve incoming requests.

sudo a2enmod mpm_event-----apache2 enable modification multi processing module event

Apache allows only one MPM to be active at a time

sudo a2dismod mpm_prefork ---- apache2 disable modification mpm prefork

(sudo a2enconf php7.4-fpm)--apache2 enable config php 7.4 - fast cgi process manager. It helps Apache communicate with PHP more efficiently and flexibly. PHP-FPM (FastCGI Process Manager) is a process manager for PHP that separates PHP processing from the web server.

Adjust ServerTokens and ServerSignature:
ServerTokens is a configuration directive that controls what information about the server is included in the server response headers. There are several types of tokens available but prod is the most minimal option, providing only the "Server: Server" response header without any version information.

The purpose of configuring ServerTokens is to control the amount of information disclosed to potential attackers.

So without a server token, it will display like Server: Apache/2.4.41 (Unix), but after using the server token it will be like Server: Server.

ServerSignature is an Apache configuration directive that determines whether the server includes a footer line with information about the server software and version in its HTTP response headers.

So when ServerSignature is set to "Off," Apache does not include the server information in error pages. This is considered a security best practice because it helps to limit the amount of information exposed to potential attackers

1. Edit Apache Configuration File:
sudo vi /etc/apache2/conf-available/security.conf
----------Configuration available , Security Configuration.

2. Adjust ServerTokens and ServerSignature:
ServerTokens Prod
ServerSignature Off

Install Caching Modules:

The cache is a foundational module that provides a caching framework, while cache_disk is a specific storage module that works with cache to implement disk-based caching. When you enable both modules, you set up the infrastructure needed for Apache to cache content on the disk, improving performance by serving cached content instead of regenerating it for each request.

sudo a2enmod cache
sudo a2enmod cache_disk

Don’t forget to restart the apache2 server to apply changes to the Apache configuration.

sudo systemctl restart apache2
sudo systemctl restart httpd

PHP ---------- PHP stands for Hypertext Preprocessor, its original name is Personal Home Page. It's a scripting language that is mainly used for web development. Imagine you're building a website, and you want it to do things like display dynamic content, handle forms, or interact with a database. That's where PHP comes in.

PHP is a server-side scripting language that runs on the web server

rather than in the user's browser. This enables server-side processing, allowing websites to perform actions before delivering content to users. For example, you can use PHP to pull information from a database and display it on a webpage.

PHP Tuning

PHP tuning involves adjusting various configuration parameters to optimize the performance and behavior of PHP on a web server.

Open the PHP configuration file:
cd /etc/php/7.4/apache2
vi php.ini

```
; Decides whether PHP may expose the fact that it is installed on the server
; (e.g. by adding its signature to the Web server header).  It is no security
; threat in any way, but it makes it possible to determine whether you use PHP
; on your server or not.
; http://php.net/expose-php
expose_php = On

;;;;;;;;;;;;;;;;;;;
; Resource Limits ;
;;;;;;;;;;;;;;;;;;;

; Maximum execution time of each script, in seconds
; http://php.net/max-execution-time
; Note: This directive is hardcoded to 0 for the CLI SAPI
max_execution_time = 30

; Maximum amount of time each script may spend parsing request data. It's a good
; idea to limit this time on productions servers in order to eliminate unexpectedly
; long running scripts.
; Note: This directive is hardcoded to -1 for the CLI SAPI
; Default Value: -1 (Unlimited)
; Development Value: 60 (60 seconds)
; Production Value: 60 (60 seconds)
; http://php.net/max-input-time
max_input_time = 60
```

; Maximum input variable nesting level
; http://php.net/max-input-nesting-level
;max_input_nesting_level = 64

; How many GET/POST/COOKIE input variables may be accepted
; max_input_vars = 1000

; Maximum amount of memory a script may consume (128MB)
; http://php.net/memory-limit
memory_limit = 128M

Genereate self signed certificate:

sudo mkdir /etc/httpd/ssl ----------etc/httpd/ssl

sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/httpd/ssl/your_domain.key -out /etc/httpd/ssl/your_domain.crt

How SSL/TLS Works in Brief:

When a client (e.g., a web browser) connects to your website over HTTPS, the server sends the public SSL certificate to the client.
The client uses the certificate to authenticate the server and establish an encrypted connection.
The private key is used by the server to decrypt data sent from the client and to encrypt data sent to the client.

This directive points to the private key file associated with the SSL certificate.
The file /etc/httpd/ssl/your_domain.key contains the private key that is used to establish a secure and encrypted connection between the server and the client.
The private key must be kept secure and should not be shared with anyone.----------/etc/httpd/ssl/domain key

---

Install apache
sudo yum install -y httpd
sudo systemctl start httpd
sudo systemctl enable httpd
---

for installing php on CentOS/RHEL/AWS Linux:

Enable EPEL and Remi repositories

sudo yum install -y epel-release
sudo yum install -y https://rpms.remirepo.net/enterprise/remi-release-7.rpm

sudo vim /etc/httpd/conf/httpd.conf ---------- server hardening.

---
installing firewall
sudo yum install -y firewalld

sudo systemctl start firewalld
sudo systemctl enable firewalld

---

sudo usermod -aG groupname newuser

getent group

```
May 21 04:26:55 ip-172-31-24-25.eu-north-1.compute.internal systemd[1]: Starting The Apache HTTP Server...
May 21 04:26:56 ip-172-31-24-25.eu-north-1.compute.internal systemd[1]: Started The Apache HTTP Server.
[root@ip-172-31-24-25 ec2-user]# sudo ufw allow 80
sudo: ufw: command not found
[root@ip-172-31-24-25 ec2-user]# sudo systemctl status firewalld
● firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2024-05-21 04:26:05 UTC; 3min 16s ago
     Docs: man:firewalld(1)
 Main PID: 1824 (firewalld)
    Tasks: 2
   Memory: 24.1M
   CGroup: /system.slice/firewalld.service
           └─1824 /usr/bin/python -Es /usr/sbin/firewalld --nofork --nopid

May 21 04:26:19 ip-172-31-24-25.eu-north-1.compute.internal firewalld[1824]: WARNING: COMMAND_FAILED: '/usr/sbin/iptables
May 21 04:26:19 ip-172-31-24-25.eu-north-1.compute.internal firewalld[1824]: WARNING: COMMAND_FAILED: '/usr/sbin/iptables
May 21 04:26:19 ip-172-31-24-25.eu-north-1.compute.internal firewalld[1824]: WARNING: COMMAND_FAILED: '/usr/sbin/iptables
May 21 04:26:20 ip-172-31-24-25.eu-north-1.compute.internal firewalld[1824]: WARNING: COMMAND_FAILED: '/usr/sbin/iptables
May 21 04:26:20 ip-172-31-24-25.eu-north-1.compute.internal firewalld[1824]: WARNING: COMMAND_FAILED: '/usr/sbin/iptables
May 21 04:26:20 ip-172-31-24-25.eu-north-1.compute.internal firewalld[1824]: WARNING: COMMAND_FAILED: '/usr/sbin/iptables
May 21 04:26:20 ip-172-31-24-25.eu-north-1.compute.internal firewalld[1824]: WARNING: COMMAND_FAILED: '/usr/sbin/iptables
May 21 04:26:20 ip-172-31-24-25.eu-north-1.compute.internal firewalld[1824]: WARNING: COMMAND_FAILED: '/usr/sbin/iptables
May 21 04:26:21 ip-172-31-24-25.eu-north-1.compute.internal firewalld[1824]: WARNING: COMMAND_FAILED: '/usr/sbin/iptables
May 21 04:26:21 ip-172-31-24-25.eu-north-1.compute.internal firewalld[1824]: WARNING: COMMAND_FAILED: '/usr/sbin/iptables
Hint: Some lines were ellipsized, use -l to show in full.
[root@ip-172-31-24-25 ec2-user]# sudo systemctl enable firewalld
[root@ip-172-31-24-25 ec2-user]# sudo firewall-cmd --permanent --add-service=http
success
[root@ip-172-31-24-25 ec2-user]# sudo firewall-cmd --permanent --add-service=https
success
[root@ip-172-31-24-25 ec2-user]# sudo firewall-cmd --reload
success
[root@ip-172-31-24-25 ec2-user]#
```

After installing firewalld ,

sudo yum install -y firewalld
sudo systemctl start firewalld
sudo systemctl enable firewalld
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --reload

I allowed traffic for http and https.

Configure Apache to Use MPM Prefork:

```
conf/           conf.d/         conf.modules.d/
[root@ip-172-31-24-25 ec2-user]# sudo vi /etc/httpd/conf.modules.d/00-mpm.conf
[root@ip-172-31-24-25 ec2-user]#
```

Entered here and uncommented this line,

LoadModule mpm_prefork_module modules/mod_mpm_prefork.so.

And then i entered into apache config file..

sudo vi /etc/httpd/conf/httpd.conf

And added mpm prefork parameters:

```
# Load config files in the "/etc/htt

<IfModule mpm_prefork_module>
    StartServers       5
    MinSpareServers    5
    MaxSpareServers   10
    MaxRequestWorkers  150
    MaxConnectionsPerChild   0
</IfModule>
```

Restarted and checked the status of apache.

I have tested apache performance using apache bench

ab -n 1000 -c 10 http://localhost/

```
IncludeOptional conf.d/*.conf
[root@ip-172-31-24-25 ec2-user]# ab -n 1000 -c 10 http://localhost/
This is ApacheBench, Version 2.3 <$Revision: 1913912 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Completed 600 requests
Completed 700 requests
Completed 800 requests
Completed 900 requests
Completed 1000 requests
Finished 1000 requests


Server Software:        Apache/2.4.59
Server Hostname:        localhost
Server Port:            80

Document Path:          /
Document Length:        3630 bytes

Concurrency Level:      10
Time taken for tests:   0.169 seconds
Complete requests:      1000
Failed requests:        0
Non-2xx responses:      1000
Total transferred:      3932000 bytes
HTML transferred:       3630000 bytes
Requests per second:    5934.05 [#/sec] (mean)
Time per request:       1.685 [ms] (mean)
Time per request:       0.169 [ms] (mean, across all concurrent requests)
Transfer rate:          22785.82 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.1      0       1
Processing:     1    2   0.4      2       4
Waiting:        1    1   0.3      1       4
Total:          1    2   0.4      2       4

Percentage of the requests served within a certain time (ms)
  50%      2
  66%      2
  75%      2
  80%      2
  90%      2
  95%      2
  98%      3
  99%      3
 100%      4 (longest request)
[root@ip-172-31-24-25 ec2-user]#
```

- Results of the test:
  - Requests per second: 5934.05 [#/sec] (mean)
  - Time per request: 1.685 [ms] (mean)
  - Transfer rate: 22785.82 [Kbytes/sec] received
  - All requests completed successfully with no failed requests.

For MPM Event handling:

Uncommented mpm event modules line

```
# one of the following LoadModule lines.  See the httpd.service(8) man
# page for more information on changing the MPM.

# prefork MPM: Implements a non-threaded, pre-forking web server
# See: http://httpd.apache.org/docs/2.4/mod/prefork.html
#
# NOTE: If enabling prefork, the httpd_graceful_shutdown SELinux
# boolean should be enabled, to allow graceful stop/shutdown.
#
#LoadModule mpm_prefork_module modules/mod_mpm_prefork.so

# worker MPM: Multi-Processing Module implementing a hybrid
# multi-threaded multi-process web server
# See: http://httpd.apache.org/docs/2.4/mod/worker.html
#
#LoadModule mpm_worker_module modules/mod_mpm_worker.so

# event MPM: A variant of the worker MPM with the goal of consuming
# threads only for connections with active processing
# See: http://httpd.apache.org/docs/2.4/mod/event.html
#
LoadModule mpm_event_module modules/mod_mpm_event.so

[root@ip-172-31-24-25 conf.modules.d]#
```

I went into httpd.conf file and added the following lines,

```
<IfModule mod_http2.c>
    Protocols h2 h2c http/1.1
</IfModule>


# Supplemental configuration
#
# Load config files in the "/etc/httpd/conf.d" directory, if any.
IncludeOptional conf.d/*.conf

<IfModule mpm_event_module>
    StartServers          2
    MinSpareThreads      25
    MaxSpareThreads      75
    ThreadLimit          64
    ThreadsPerChild      25
    MaxRequestWorkers   150
    MaxConnectionsPerChild   0
</IfModule>

[root@ip-172-31-24-25 conf]#
```

\

And then i have restarted the httpd server again.

```
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Completed 600 requests
Completed 700 requests
Completed 800 requests
Completed 900 requests
Completed 1000 requests
Finished 1000 requests


Server Software:        Apache/2.4.59
Server Hostname:        localhost
Server Port:            80

Document Path:          /
Document Length:        3630 bytes

Concurrency Level:      10
Time taken for tests:   0.190 seconds
Complete requests:      1000
Failed requests:        0
Non-2xx responses:      1000
Total transferred:      3921000 bytes
HTML transferred:       3630000 bytes
Requests per second:    5251.60 [#/sec] (mean)
Time per request:       1.904 [ms] (mean)
Time per request:       0.190 [ms] (mean, across all concurrent requests)
Transfer rate:          20108.93 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.1      0       1
Processing:     0    2   1.3      2      27
Waiting:        0    2   1.0      1      18
Total:          0    2   1.3      2      27

Percentage of the requests served within a certain time (ms)
  50%      2
  66%      2
  75%      2
  80%      2
  90%      3
  95%      4
  98%      5
  99%      5
 100%     27 (longest request)
[root@ip-172-31-24-25 conf.modules.d]#
```

And then i did apache bench.

For checking active MPM:

```
 100%     27 (longest request)
[root@ip-172-31-24-25 conf.modules.d]# sudo apachectl -M | grep mpm
 mpm_event_module (shared)
[root@ip-172-31-24-25 conf.modules.d]#
[root@ip-172-31-24-25 conf.modules.d]#
```

Dockerize PHP-FPM and related services (Apache httpd Server) using Docker Compose or Kubernetes for easier deployment, scaling, and management of PHP applications.Explore container orchestration platforms like Kubernetes to automate PHP-FPM deployments, manage configurations with ConfigMaps or Secrets, and utilize features like horizontal scaling based on traffic demand.

```
[root@ip-172-31-24-25 php-app]# ls
docker-compose.yml  Dockerfile  index.php  public
[root@ip-172-31-24-25 php-app]# cat docker-compose.yml
version: '3.8'
```

Created php-app project directory.

```
[root@ip-172-31-24-25 php-app]# cat docker-compose.yml
version: '3.8'

services:
  php-fpm:
    build: .
    ports:
      - "9000:9000"
    volumes:
      - ./public:/var/www/html

  httpd:
    image: httpd:2.4
    ports:
      - "8081:80"
    volumes:
      - ./public:/usr/local/apache2/htdocs/

[root@ip-172-31-24-25 php-app]#
```

Created docker-compose.yml file

```
root@ip-172-31-24-25 php-app]# cat Dockerfile
FROM php:7.4-fpm

# Copy application files
COPY public /var/www/html

# Set working directory
WORKDIR /var/www/html

# Expose port 9000
EXPOSE 9000

root@ip-172-31-24-25 php-app]#
```

Created Dockerfile

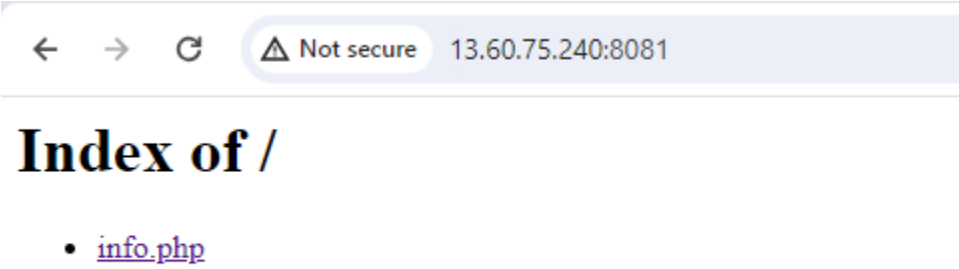
```
[root@ip-172-31-24-25 php-app]# cat index.php
<?php
phpinfo();

[root@ip-172-31-24-25 php-app]# cd public/
[root@ip-172-31-24-25 public]# ls
info.php
[root@ip-172-31-24-25 public]# cat info.php
<?php phpinfo(); ?>
[root@ip-172-31-24-25 public]#
```

Did docker-compose up command

```
[root@ip-172-31-24-25 public]# cat info.php
<?php phpinfo(); ?>
[root@ip-172-31-24-25 public]# docker-compose up
Starting php-app_php-fpm_1 ... done
Starting php-app_httpd_1   ... done
Attaching to php-app_php-fpm_1, php-app_httpd_1
php-fpm_1  | [21-May-2024 11:00:53] NOTICE: fpm is running, pid 1
php-fpm_1  | [21-May-2024 11:00:53] NOTICE: ready to handle connections
httpd_1    | AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.20.0.3. Set the 'Serve
httpd_1    | AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.20.0.3. Set the 'Serve
httpd_1    | [Tue May 21 11:00:53.473068 2024] [mpm_event:notice] [pid 1:tid 139697482717056] AH00489: Apache/2.4.59 (Unix) configur
httpd_1    | [Tue May 21 11:00:53.473714 2024] [core:notice] [pid 1:tid 139697482717056] AH00094: Command line: 'httpd -D FOREGROUND
```

Tried accessing the php app with server ip with port number



```
[root@ip-172-31-24-25 php-app]# docker-compose up
php-app_php-fpm_1 is up-to-date
php-app_httpd_1 is up-to-date
Attaching to php-app_php-fpm_1, php-app_httpd_1
httpd_1    | AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.22.0.3. Set the '
httpd_1    | AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.22.0.3. Set the '
httpd_1    | [Tue May 21 11:09:41.940066 2024] [mpm_event:notice] [pid 7:tid 140677389399936] AH00489: Apache/2.4.59 (Unix) con
httpd_1    | [Tue May 21 11:09:41.940385 2024] [core:notice] [pid 7:tid 140677389399936] AH00094: Command line: 'httpd -D FOREG
httpd_1    | 196.12.58.69 - - [21/May/2024:11:09:49 +0000] "GET /info.php HTTP/1.1" 304 -
httpd_1    | 196.12.58.69 - - [21/May/2024:11:09:51 +0000] "GET /info.php HTTP/1.1" 304 -
httpd_1    | 196.12.58.69 - - [21/May/2024:11:10:00 +0000] "GET / HTTP/1.1" 200 203
httpd_1    | 196.12.58.69 - - [21/May/2024:11:10:52 +0000] "-" 408 -
httpd_1    | 196.12.58.69 - - [21/May/2024:11:16:35 +0000] "GET /index.php HTTP/1.1" 404 196
php-fpm_1  | [21-May-2024 11:09:41] NOTICE: fpm is running, pid 1
php-fpm_1  | [21-May-2024 11:09:41] NOTICE: ready to handle connections
httpd_1    | 196.12.58.69 - - [21/May/2024:11:17:03 +0000] "GET /index.php HTTP/1.1" 404 196
httpd_1    | 196.12.58.69 - - [21/May/2024:11:17:05 +0000] "GET /index.php HTTP/1.1" 404 196
httpd_1    | 196.12.58.69 - - [21/May/2024:11:17:12 +0000] "GET / HTTP/1.1" 200 203
^CGracefully stopping... (press Ctrl+C again to force)
Stopping php-app_httpd_1   ... done
Stopping php-app_php-fpm_1 ... done
[root@ip-172-31-24-25 php-app]#
```

```
sudo yum install python3 augeas-libs
```

```
sudo python3 -m venv /opt/certbot/
```

```
sudo /opt/certbot/bin/pip install --upgrade pip
```

```
sudo /opt/certbot/bin/pip install certbot certbot-apache
```

```
sudo ln -s /opt/certbot/bin/certbot /usr/bin/certbot
```